

# Universidad Nacional Autonoma de México.

## Facultad de Estudios Superiores Acatlán.

### Tecnicas Estadisticas y Minería de Datos

#### *Modulo IV*

---

## *Practica: Preprocesamiento de los datos*

---

**Profesor:** JAVIER ROSAS HERNÁNDEZ \

**Grupo - 35**

**15**

**de Junio del 2025**

Antes de comenzar hay que establecer algunas paqueterías necesarios para realizar cada uno de los incisos y de paso el exporta la información para su previa manipulación.

```
In [37]: import numpy as np      # Manipulacion de estructuras matematicas
import pandas as pd      # Manipulacion de base de datos en tablas
import seaborn as sns    # Creacion de graficas estadisticas
import matplotlib.pyplot as plt    # Creacion de las graficas descriptivas en genera
```

## Primer Problema

Supongamos que los valores de un conjunto determinado de datos se agrupan en intervalos. Los intervalos y las frecuencias correspondientes son los siguientes:

| Edad   | Frecuencia |
|--------|------------|
| 1-5    | 550        |
| 5-15   | 950        |
| 15-20  | 1950       |
| 20-50  | 750        |
| 50-80  | 600        |
| 80-110 | 180        |

Calcular un valor medio aproximado para los datos.

### Solucion:

Vamos almacenar los datos del ejercicio en un tabla; pero vamos a analizar la situacion. Recordemos que **el valor medio** no es mas que un promedio de los datos; el detalle recai en el hecho de que las frecuencias son extraidos der un intervalo y por ende no podemos establecer un numero de observaciones.

Para ello lo vamos a sustituir por **el promedio ponderado** que es practicamente lo mismo, solo los datos tiene un pesos  $w_i$  ante los demas y lo podemos ver como:

$$\frac{\sum w_i x_i}{\sum x_i}$$

Para el caso de los intervalos, dicho pesos  $w_i$  se usara **el valor del medio...**

```
In [38]: # Creamos la Lista que almacene los datos
lista = [['1 - 5',550],['5 - 15',950],['15 - 20',1950],['20 - 50',750],['50 - 80',600],['80 - 110',180]]

# Convertimos este objeto en tabla con la columna
Edades = pd.DataFrame(lista, columns = ['Edad', 'Frecuencias'])

#Visualizamos la tabla
Edades
```

```
Out[38]:
```

|   | Edad     | Frecuencias |
|---|----------|-------------|
| 0 | 1 - 5    | 550         |
| 1 | 5 - 15   | 950         |
| 2 | 15 - 20  | 1950        |
| 3 | 20 - 50  | 750         |
| 4 | 50 - 80  | 600         |
| 5 | 80 - 110 | 180         |

```
In [39]: # Agregamos la columna de valor medio de cada intervalo
Edades['Valor Medio'] = [3,10,17,35,65,95]

# Ordenamos para mejor presentacion
Edades = Edades[['Edad', 'Valor Medio', 'Frecuencias']]

# Visualizamos la tabla
Edades
```

```
Out[39]:
```

|   | Edad     | Valor Medio | Frecuencias |
|---|----------|-------------|-------------|
| 0 | 1 - 5    | 3           | 550         |
| 1 | 5 - 15   | 10          | 950         |
| 2 | 15 - 20  | 17          | 1950        |
| 3 | 20 - 50  | 35          | 750         |
| 4 | 50 - 80  | 65          | 600         |
| 5 | 80 - 110 | 95          | 180         |

Nadamas pasamos a realizar el promedio....

```
In [40]: # Realizamos la suma en ambas partes de la division...

dividendo = (Edades['Valor Medio'] * Edades['Frecuencias']).sum()
divisor = Edades['Frecuencias'].sum()

# Calculamos el promedio
print(f"El valor medio de los datos sera: {(dividendo / divisor).round(3)}")
```

El valor medio de los datos sera: 25.432

## Segundo Problema

Supongamos que los datos para el análisis incluyen el atributo de Edad. Los valores de la edad para las tuplas de datos son (en orden creciente) 12, 15, 15, 16, 19, 19, 20, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 34, 35, 35, 35, 35, 36, 36, 36, 42, 44, 45, 46, 52, 71, 76, 78

- ¿Cuál es la media de los datos? ¿Cuál es la mediana?
- ¿Cuál es la moda de los datos?
- Utilice la normalización min-max para transformar el valor 50 de la antigüedad en el intervalo [0.0,1.0].
- Trazar un histograma de equi-distancia de longitud 8
- Determina visualmente si hay datos anómalos

## Solucion:

Vamos crear una lista que almacene los datos y despues en una tabla (*para mayor comodidad*).

```
In [41]: # Creamos la lista y los ordenamos
D_V = [12,16,21,30,34,42,44,45,46,52,71,76,78] + [15]*2 + [19]*2 + [20]*3 + [22]*2
D_V = sorted(D_V)

# Visualizamos la lista
print(D_V)

# Convertimos este objeto en tabla con la columna
D_V = pd.DataFrame(D_V, columns = ['Edad'])
```

[12, 15, 15, 16, 19, 19, 20, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 34, 35, 35, 35, 36, 36, 36, 42, 44, 45, 46, 52, 71, 76, 78]

Para el **inciso a) y b)** lo podemos resolver por el siguiente codigo...

```
In [42]: # Para la media
media1 = D_V['Edad'].mean()

# Para la mediana
mediana1 = D_V['Edad'].median()

# Para la moda
moda1 = D_V['Edad'].mode()

# Visualizamos los resultados
print("Podemos concluir que:")
print(f"la media es: {media1.round(4)}")
print(f"la mediana: {int(mediana1)}")
print(f"la moda: {moda1.values}")
```

Podemos concluir que:

la media es: 32.9429

la mediana: 33

la moda: [25 35]

Para el **inciso c)** vamos agregar una nueva columna a la tabla base con **los datos normalizacion**.

Recordemos se aplica la siguiente formula:

$$\frac{x_i - \text{Min}(x_i)}{\text{Max}(x_i) - \text{Min}(x_i)}$$

El codigo quedaria como:

```
In [43]: D_V['Normalizacion'] = ((D_V['Edad'] - D_V['Edad'].min()) / (D_V['Edad'].max() - D_V['Edad'].min()))
```

```
# Visualizamos Los datos
D_V.head(8)
```

```
Out[43]:
```

|   | Edad | Normalizacion |
|---|------|---------------|
| 0 | 12   | 0.0000        |
| 1 | 15   | 0.0455        |
| 2 | 15   | 0.0455        |
| 3 | 16   | 0.0606        |
| 4 | 19   | 0.1061        |
| 5 | 19   | 0.1061        |
| 6 | 20   | 0.1212        |
| 7 | 20   | 0.1212        |

```
In [44]: # Veamos que Los extremos sean 0 y 1
Normal_min = D_V['Normalizacion'] == D_V['Normalizacion'].min()
Normal_max = D_V['Normalizacion'] == D_V['Normalizacion'].max()

# Visualizamos Los datos
print(D_V[Normal_min])
print(D_V[Normal_max])
```

```
Edad Normalizacion
0    12           0.0
Edad Normalizacion
34   78           1.0
```

Para finalizar este inciso, vamos a ver como quedaria normalizado el dato 50 y dado a que este no es ni el minimo, ni el maximo en el conjunto de datos...

```
In [45]: Dato_50 = (50 - D_V['Edad'].min()) / (D_V['Edad'].max() - D_V['Edad'].min())
Dato_50
```

```
Out[45]: 0.5757575757575758
```

Para el caso de la grafica del histograma del **inciso d)**, sera de 2 forma distintas:

- Para los datos originales

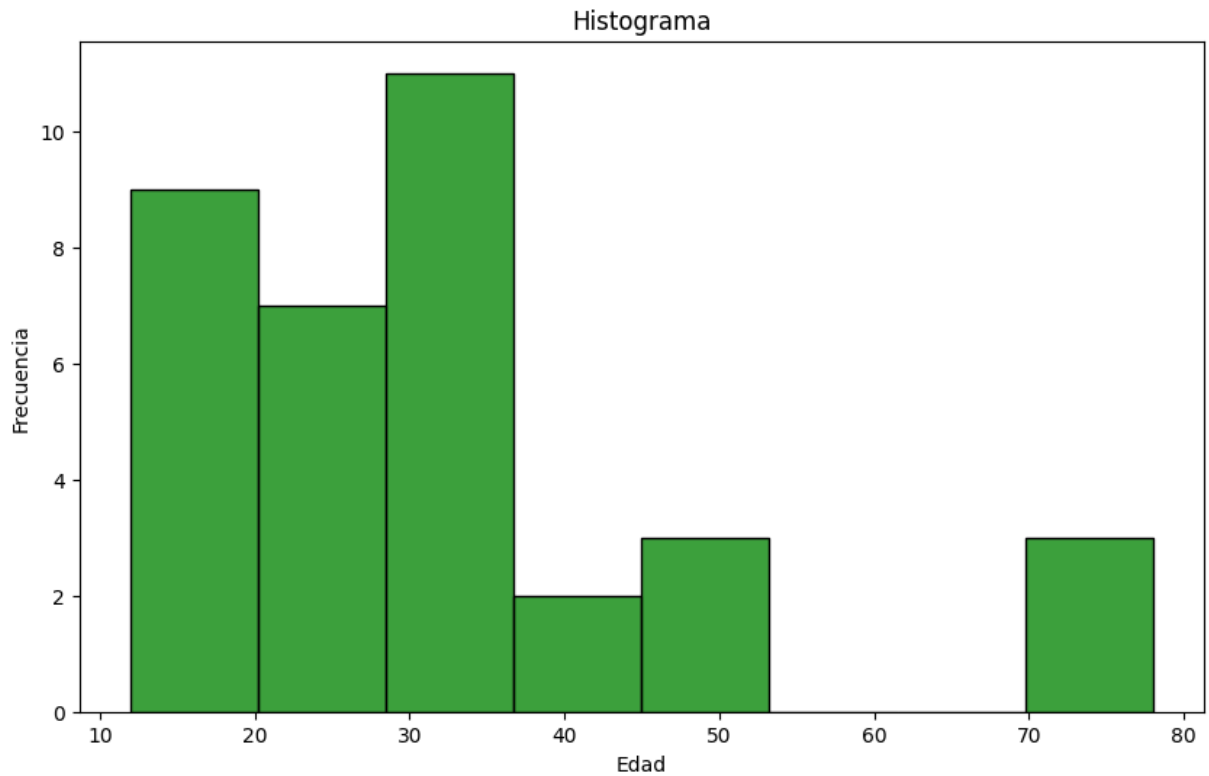
```
In [46]: # Establecemos el tamaño del lienzo
plt.figure(figsize = (10, 6))

# Codificamos Los datos en el histograma
sns.histplot(D_V['Edad'], bins = 8, kde = False, color = 'green')

# Configuramos Los elemento
plt.title("Histograma")
plt.xlabel("Edad")
```

```
plt.ylabel("Frecuencia")

# Visualizamos la grafica
plt.show()
```



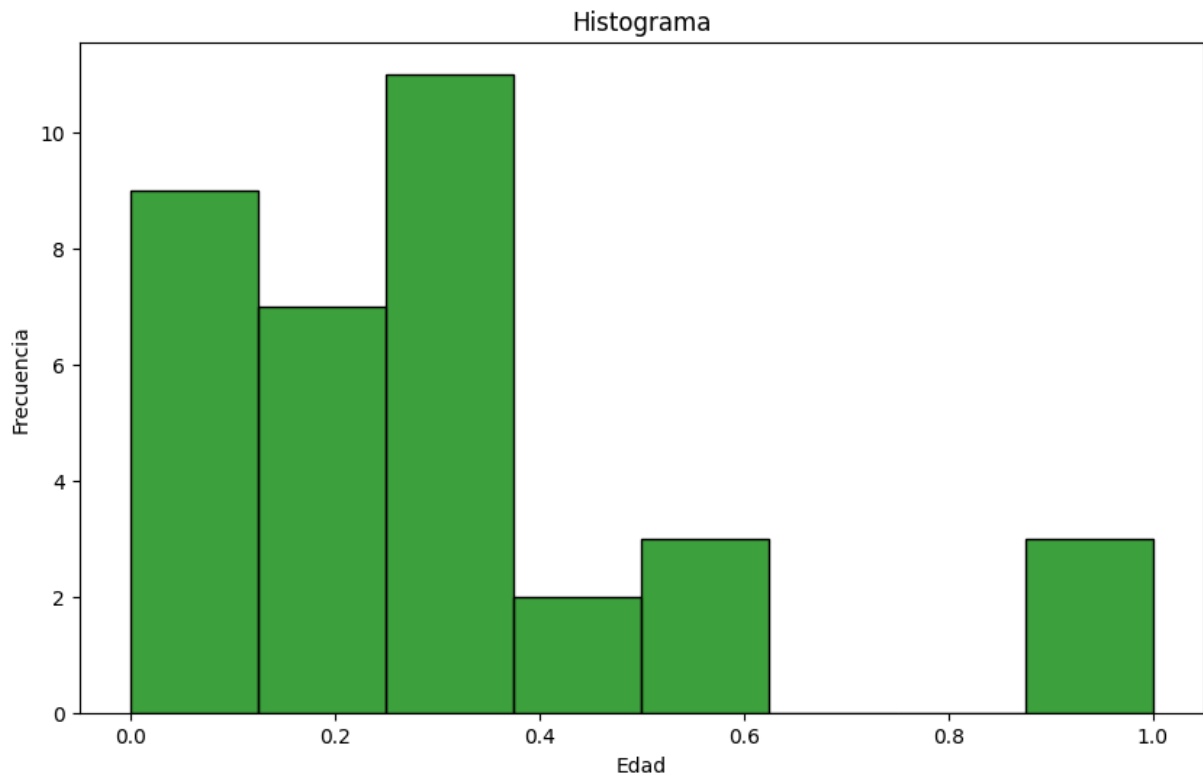
- Para los datos Normalizados

```
In [47]: # Establecemos el tamaño del lienzo
plt.figure(figsize = (10, 6))

# Codificamos los datos en el histograma
sns.histplot(D_V['Normalizacion'], bins = 8, kde = False, color = 'green')

# Configuramos los elementos
plt.title("Histograma")
plt.xlabel("Edad")
plt.ylabel("Frecuencia")

# Visualizamos la grafica
plt.show()
```



Es practicamente la misma, con la diferencia es que los datos son mas reducidos...

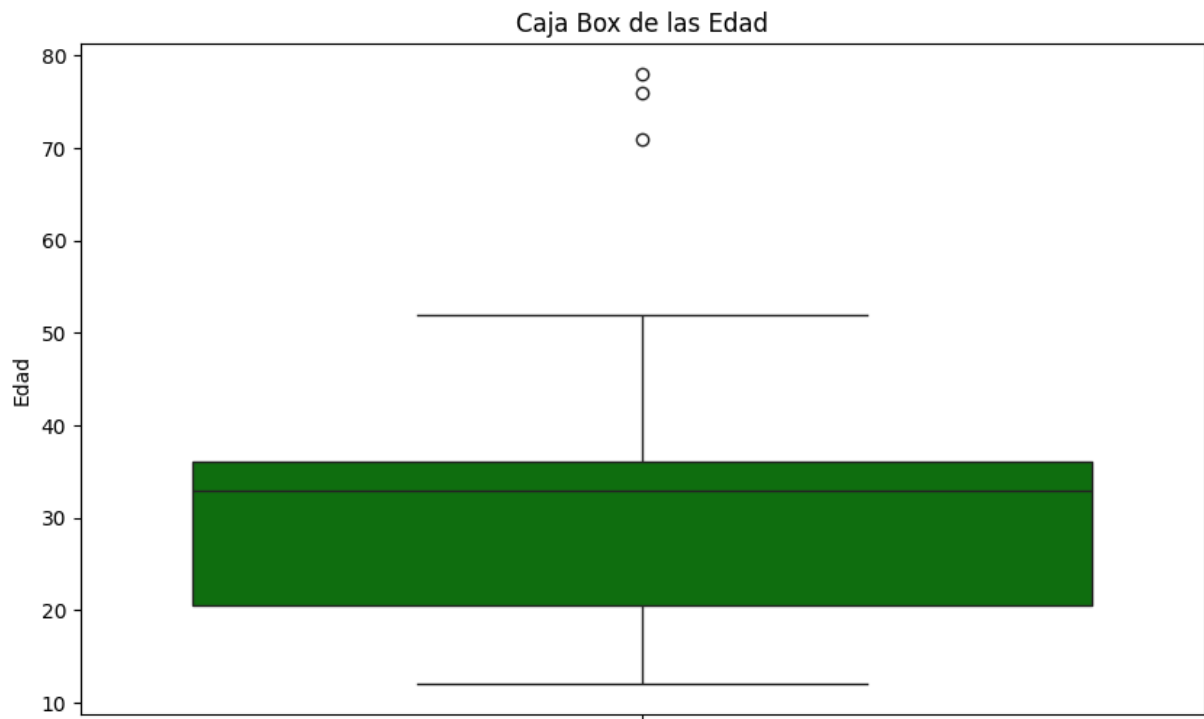
Podemos detectar que algunos datos esta lejos de aquellos que estan acumulados (*indicio de datos anomalos*). Para corroborar si hay o no **Datos Anomalos** (*inciso e*) lo veremos **una grafica Box...**

```
In [48]: # Establecemos el tamaño del lienzo
plt.figure(figsize = (10, 6))

# Codificamos los datos en la grafica box
sns.boxplot(D_V['Edad'], color = 'green')

# Configuramos los elementos
plt.title("Caja Box de las Edad")
plt.ylabel("Edad")

# Visualizamos la grafica
plt.show()
```



Podemos detectar que hay 3 valores que esta fuera del conjunto y veamos cuales son estos datos (*curiosamente son los ultimos datos*).

In [49]: `D_V.tail(5)`

Out[49]:

|           | Edad | Normalizacion |
|-----------|------|---------------|
| <b>30</b> | 46   | 0.5152        |
| <b>31</b> | 52   | 0.6061        |
| <b>32</b> | 71   | 0.8939        |
| <b>33</b> | 76   | 0.9697        |
| <b>34</b> | 78   | 1.0000        |

De los ultimos 3 registros podemos ver que hay un salto de valores; y por ende, son distinto a la mayoría de los demas.

## Tercer Problema



Supongamos que un hospital probó los datos de edad y grasa corporal para 18 adultos seleccionados al azar con el siguiente resultado

|         |     |      |     |      |      |      |      |      |      |
|---------|-----|------|-----|------|------|------|------|------|------|
| Edad    | 22  | 23   | 27  | 28   | 39   | 42   | 47   | 50   | 50   |
| % Grasa | 9.5 | 27.5 | 7.7 | 18.8 | 31.4 | 26.9 | 27.5 | 27.2 | 31.3 |

|         |      |      |      |      |      |      |      |      |      |
|---------|------|------|------|------|------|------|------|------|------|
| Edad    | 52   | 54   | 55   | 56   | 57   | 59   | 60   | 62   | 66   |
| % Grasa | 35.5 | 42.5 | 28.9 | 33.5 | 31.2 | 34.1 | 33.3 | 41.4 | 36.7 |

- Calcule la media, la mediana y la desviación estándar de la edad y % Grasa.
- Grafica el diagrama de dispersión
- Normaliza los datos de Edad y % Grasa
- Determina visualmente si hay datos atípicos
- Calcula y explica el coeficiente de correlación de Pearson

**Solucion:**

Almacenamos los datos en un objeto en una tabla *(para mejor comodidad)*

```
In [50]: # Creamos la Lista con Los datos
Med_G = [(22,9.5),(23,27.5),(27,7.7),(28,18.8),(39,31.4),(42,26.9),(47,27.5),(50,27.2),(52,35.5),(54,42.5),(55,28.9),(56,33.5),(57,31.2),(59,34.1),(60,33.3),(62,41.4),(66,36.7)]

# Convertimos este objeto en tabla con la columna
Med_G = pd.DataFrame(Med_G, columns = ['Edad', 'Grasa'])

# Visualizamos la tabla
Med_G
```

Out[50]:

|           | Edad | Grasa |
|-----------|------|-------|
| <b>0</b>  | 22   | 9.5   |
| <b>1</b>  | 23   | 27.5  |
| <b>2</b>  | 27   | 7.7   |
| <b>3</b>  | 28   | 18.8  |
| <b>4</b>  | 39   | 31.4  |
| <b>5</b>  | 42   | 26.9  |
| <b>6</b>  | 47   | 27.5  |
| <b>7</b>  | 50   | 27.2  |
| <b>8</b>  | 50   | 31.3  |
| <b>9</b>  | 52   | 35.5  |
| <b>10</b> | 54   | 42.5  |
| <b>11</b> | 55   | 28.9  |
| <b>12</b> | 56   | 33.5  |
| <b>13</b> | 57   | 31.2  |
| <b>14</b> | 59   | 34.1  |
| <b>15</b> | 60   | 33.3  |
| <b>16</b> | 62   | 41.4  |
| <b>17</b> | 66   | 36.7  |

Para el calculo de los terminos que pide el **inciso a)** lo haremos de forma resumida con el siguiente comando:

```
In [51]: # Calculamos los terminos estadisticos clasico
Result = Med_G.describe()
Result
```

```
Out[51]:
```

|              | Edad      | Grasa     |
|--------------|-----------|-----------|
| <b>count</b> | 18.000000 | 18.000000 |
| <b>mean</b>  | 47.166667 | 29.161111 |
| <b>std</b>   | 13.904083 | 9.292194  |
| <b>min</b>   | 22.000000 | 7.700000  |
| <b>25%</b>   | 39.750000 | 27.275000 |
| <b>50%</b>   | 51.000000 | 31.250000 |
| <b>75%</b>   | 56.750000 | 33.950000 |
| <b>max</b>   | 66.000000 | 42.500000 |

```
In [52]: # Calculamos los terminos estadisticos clasico
Result = Med_G.describe()

# Quitamos las filas que no es de nuestro interes
Result.drop(['count', 'min', 'max', '25%', '75%'], inplace = True)

# Cambiamos los indices de los registros para mejor entendimiento.
Result.index = ['Media', 'Desviacion Estandar', 'Mediana (Percentil 50%)']

# Visualizamos la tabla
Result.round(4)
```

```
Out[52]:
```

|                                | Edad    | Grasa   |
|--------------------------------|---------|---------|
| <b>Media</b>                   | 47.1667 | 29.1611 |
| <b>Desviacion Estandar</b>     | 13.9041 | 9.2922  |
| <b>Mediana (Percentil 50%)</b> | 51.0000 | 31.2500 |

La **Grafica de dispersion** nos ayuda a resolver el **inciso b)** y a ver la tendencia de los datos que disponemos...

```
In [53]: # Establecemos el tamaño de la grafica
plt.figure(figsize = (10, 6))

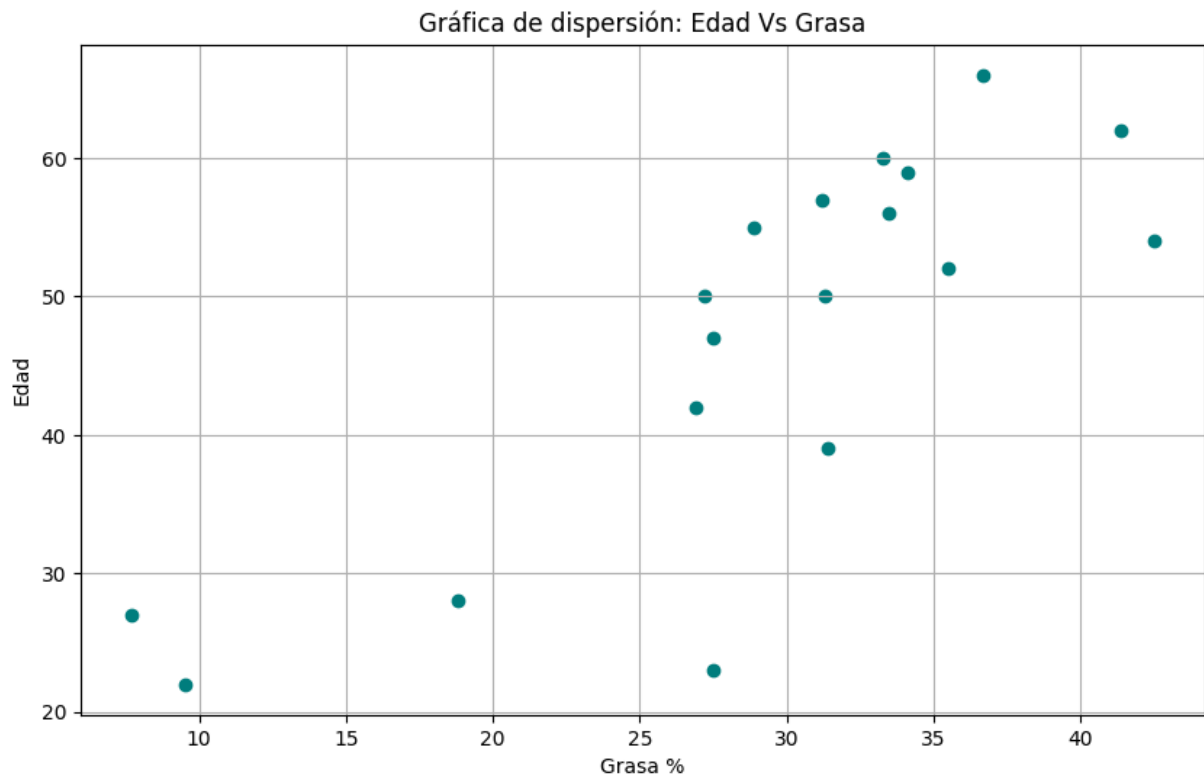
# Indicamos los ejes
y = Med_G['Edad']
x = Med_G['Grasa']

# Codificamos la grafica de dispersion para la columna "D"
plt.scatter(x, y, color = 'teal')

# Configuramos los elementos de la grafica
plt.title('Gráfica de dispersión: Edad Vs Grasa')
plt.xlabel('Grasa %')
```

```
plt.ylabel('Edad')

#Visualizamos la misma
plt.grid(True)
plt.show()
```



Podemos observar que hay una acumulacion de datos en la parte superior derecha (*indicio datos atipicos por su contraparte*) y train una tendencia positiva e creciente...

Por otra parte; como lo indica el **inciso c)**, veamos como nos queda los datos normalizados y ver la comparativa en la misma tabla.

```
In [54]: # Realizamos los calculos
Med_G['Norm_Edad'] = ((Med_G['Edad'] - Med_G['Edad'].min()) / (Med_G['Edad'].max()
Med_G['Norm_Grasa'] = ((Med_G['Edad'] - Med_G['Edad'].min()) / (Med_G['Edad'].max()

# Ordenamos la tabla para entender mejor las comparativa
Med_G = Med_G[['Edad', 'Norm_Edad', 'Grasa', 'Norm_Grasa']]

# Visualizamos los datos
Med_G.head(8)
```

```
Out[54]:
```

|   | Edad | Norm_Edad | Grasa | Norm_Grasa |
|---|------|-----------|-------|------------|
| 0 | 22   | 0.0000    | 9.5   | 0.0000     |
| 1 | 23   | 0.0227    | 27.5  | 0.0227     |
| 2 | 27   | 0.1136    | 7.7   | 0.1136     |
| 3 | 28   | 0.1364    | 18.8  | 0.1364     |
| 4 | 39   | 0.3864    | 31.4  | 0.3864     |
| 5 | 42   | 0.4545    | 26.9  | 0.4545     |
| 6 | 47   | 0.5682    | 27.5  | 0.5682     |
| 7 | 50   | 0.6364    | 27.2  | 0.6364     |

Como en el problema anterior, vamos a buscar los registros que tengan este comportamiento **Anormal a los demas**, que son como unos 4 por lo que pudimos ver en el grafico de dispersion...

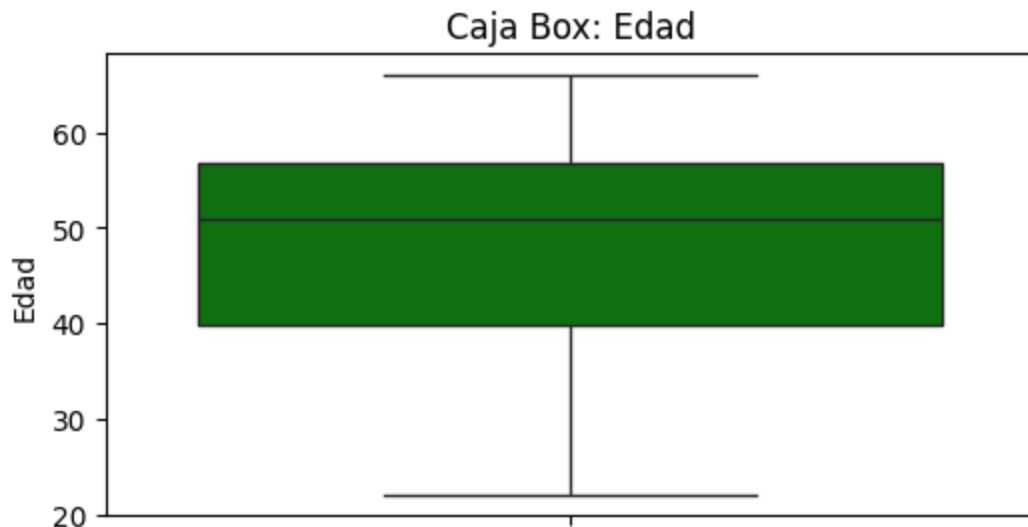
- Para el caso de la edad:

```
In [55]: # Establecemos el tamaño del lienzo
plt.figure(figsize = (6, 3))

# Codificamos los datos en la grafica box
sns.boxplot(Med_G['Edad'], color = 'green')

# Configuramos los elementos
plt.title("Caja Box: Edad")
plt.ylabel("Edad")

# Visualizamos la grafica
plt.show()
```



La grafica Box esta indicado el hecho de que este apartado, no se detecta valores anomalos.

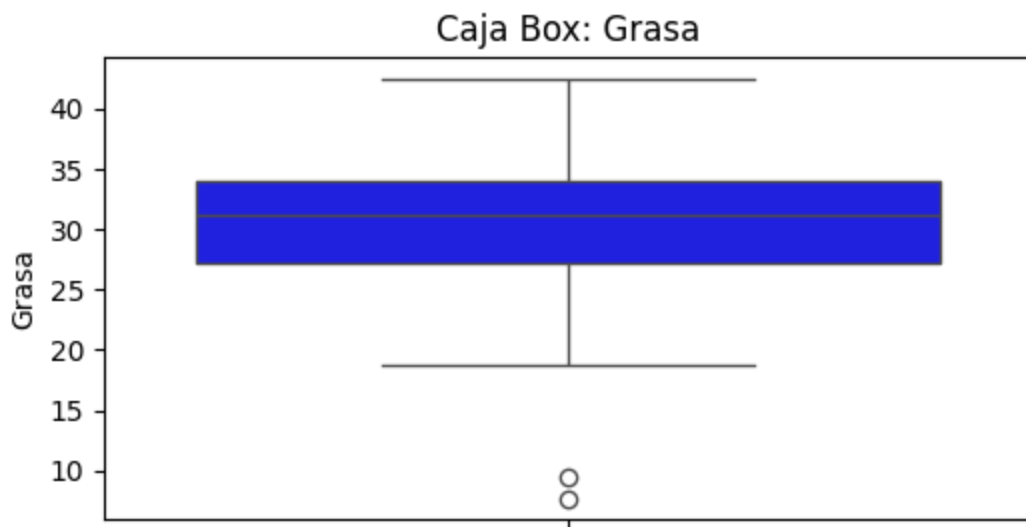
- Para el caso de la Grasa:

```
In [56]: # Establecemos el tamaño del lienzo
plt.figure(figsize = (6, 3))

# Codificamos los datos en la grafica box
sns.boxplot(Med_G['Grasa'], color = 'blue')

# Configuramos los elementos
plt.title("Caja Box: Grasa")
plt.ylabel("Grasa")

# Visualizamos la grafica
plt.show()
```



Esta ultima grafica, se da a entender que son 2 datos que pertenecen a un registro de los cuales si son anomalos y se mas conveniente el eliminarlos. Curiosamente son los valores mas pequenos de la V.A. Grasa...

```
In [57]: inf_med_G = Med_G.sort_values(by = 'Grasa')
inf_med_G.head(4)
```

```
Out[57]:
```

|   | Edad | Norm_Edad | Grasa | Norm_Grasa |
|---|------|-----------|-------|------------|
| 2 | 27   | 0.1136    | 7.7   | 0.1136     |
| 0 | 22   | 0.0000    | 9.5   | 0.0000     |
| 3 | 28   | 0.1364    | 18.8  | 0.1364     |
| 5 | 42   | 0.4545    | 26.9  | 0.4545     |

Podemos ver ese salto de valores de la columna **Grasa**; corroborando como dato atipico...

Por ultimo; para resolver el ultimo inciso (**inciso e**) daremos una explicacion de lo que es **la correlacion**.

**La correlacio:** es la relacion que existe entre 2 V.A., es decir; el cambio de valores de un conjunto de datos, afecta o altera el comportamiento de otro conjunto de datos de forma proporcional...

Por lo general; se utiliza 2 formulas de correlacion:

- **Correlacion de Pearson**
- **Correlacion de Spearman**

la diferencia entre las 2 es el hecho de que la primerra es mas eficiente a la hora de determinar la relacion entre 2 V.A. continua y la otra es eficiente sin importar el tipo de V.A. sea (*o como se comporte*)...

Veremos ambos casos ya que la variancion entre estos terminos no es demasiada. La forma en com interpretamos estos valores es la direccion que tome este termino  $\phi$  que vive entre  $[-1, 1]$

- Si  $\phi > 0$  hay una relacion proporcional de crecimiento entre ambas V.A.
- Si  $\phi < 0$  hay una relacion proporcional decreciente entre ambas V.A., es decir: si A crece, B decrece.
- Si  $\phi = 0$  No tiene relacion alguna o falta mas informacion para determinar este hecho.

Dado a que nos pide la correlacion de Pearson, lo podemos calcular con el siguiente codigo...

```
In [58]: # Correlacion de Pearson (Coctemplando los datos originales)
matriz = Med_G.drop(['Norm_Edad', 'Norm_Grasa'], axis = 1).corr(method = 'pearson')
matriz
```

```
Out[58]:
```

|       | Edad    | Grasa   |
|-------|---------|---------|
| Edad  | 1.00000 | 0.80756 |
| Grasa | 0.80756 | 1.00000 |

En conclusion, podemos decir que la relacion entre **Edad** y **Grasa** tiene relacion positiva, de la cual si una crece en una unidad, me implica que la otra crece en un 80.7

## Cuarto Problema

Cálculo de distancias entre atributos numéricos, de la siguientes tuplas:

- $X_1 = (5, 3)$

- $X_2 = (6, 10)$

Utilizando las distancias y expresar gráficamente los resultados en cada inciso:

a) Distancia maxima

Recordemos que por definicion que esta distancia establece el hecho de escojer la maxima diferencia que existe (*entre parametros*) que tenga 2 tuplas. Matematicamente se escribe como:

$$d(x_i, x_j) = \text{Max}\{|x_{ip} - x_{jp}|\}$$

**Solucion:**

$$d(x_1, x_2) = \text{Max}\{|x_{11} - x_{12}|, |x_{21} - x_{22}|\} = \text{Max}\{|5 - 6|, |3 - 10|\} \quad (1)$$

$$= \text{Max}\{|-1|, |-7|\} = \text{Max}\{1, 7\} = 7 \quad (2)$$

b) Distancia de Manhattan

Parecida a la anterior, solo se realiza una suma de dichas diferencias.

$$d(x_i, x_j) = \sum_{p=1}^n |x_{ip} - x_{jp}|$$

**Solucion:**

Con  $n = 2$ , tenemos que dicha distancia sera...

$$d(x_i, x_j) = \sum_{p=1}^2 |x_{ip} - x_{jp}| = |x_{11} - x_{12}| + |x_{21} - x_{22}| = |5 - 6| + |3 - 10| = |-1| + |-7|$$



**Solucion:**

c) Distancia de Euclidiana

Es la distancia mas usada (*matematicamente*) y se supone que es la distancia mas corta entre 2 puntos y lo podemos interpretar como:

$$d^2(\mathbf{X}_i, \mathbf{X}_j) = (\mathbf{X}_i - \mathbf{X}_j)^T (\mathbf{X}_i - \mathbf{X}_j)$$

**Solucion:**

$$d^2(\mathbf{X}_i, \mathbf{X}_j) = \begin{bmatrix} -1 \\ -7 \end{bmatrix} * [-1 \quad -7] = 1 + 49 = 50$$

$$d(\mathbf{X}_i, \mathbf{X}_j) = 3.87298$$



# Quinto Problema

Cálculo de distancias, atributos categóricos

Sea la siguiente tabla de datos

| ID | Color_ojos | Sexo | Edad |
|----|------------|------|------|
| 1  | Café       | M    | 24   |
| 2  | Negro      | M    | 23   |
| 3  | Azul       | F    | 31   |
| 4  | Negro      | F    | 27   |
| 5  | Negro      | F    | 24   |
| 6  | Verde      | M    | 28   |
| 7  | Café       | M    | 25   |

- Agrega tus datos y realiza lo siguiente
- Utilizando distancia Euclidiana, encuentre la distancia entre las tuplas 1 y 5
- Utilizando distancia Euclidiana, encuentre la distancia entre las tuplas 4 y 8
- Realiza lo mismo con la distancia de Manhattan y Mahalanobis
- Normaliza la edad

**Solucion:**

Realizando el **inciso a)** tenemos el siguiente codigo:

```
In [59]: # Creamos una serie de listas que almacene los datos
ID = [1,2,3,4,5,6,7]
Color = ['Cafe', 'Negro', 'Azul', 'Negro', 'Negro', 'Verde', 'Cafe']
Sexo = ['M', 'M', 'F', 'F', 'F', 'M', 'M']
Anos = [24,23,31,27,24,28,25]

# Unificamos los valores en una tabla
tabla = pd.DataFrame({'ID':ID, 'Color_ojos':Color, 'Sexo':Sexo, 'Edad':Anos})
tabla
```

```
Out[59]:
```

|   | ID | Color_ojos | Sexo | Edad |
|---|----|------------|------|------|
| 0 | 1  | Cafe       | M    | 24   |
| 1 | 2  | Negro      | M    | 23   |
| 2 | 3  | Azul       | F    | 31   |
| 3 | 4  | Negro      | F    | 27   |
| 4 | 5  | Negro      | F    | 24   |
| 5 | 6  | Verde      | M    | 28   |
| 6 | 7  | Cafe       | M    | 25   |

Podemos notar; que para realizar las distancia que nos pide la actividad es necesario el cambiar las V.A. Categóricas (*Color\_ojos*, *Sexo*) en valores numéricos de los cuales podemos

manipular matematicamente...

```
In [60]: # Cambiamos la columna "ID" como los indices
tabla.index = tabla['ID']

# Eliminamos la misma columna
tabla.drop('ID',axis = 1,inplace = True)

# Visualizamos los cambios
tabla
```

Out[60]:

|    | Color_ojos | Sexo | Edad |
|----|------------|------|------|
| ID |            |      |      |
| 1  | Cafe       | M    | 24   |
| 2  | Negro      | M    | 23   |
| 3  | Azul       | F    | 31   |
| 4  | Negro      | F    | 27   |
| 5  | Negro      | F    | 24   |
| 6  | Verde      | M    | 28   |
| 7  | Cafe       | M    | 25   |

```
In [61]: # Cambiamos las variables categoricas en variables numericas
tabla['Color_ojos'] = tabla['Color_ojos'].replace({'Cafe':0,'Negro':1,'Azul':2,'Verde':3})
tabla['Sexo'] = tabla['Sexo'].replace({'M':0,'F':1})

# Visualizamos los cambios
tabla
```

<ipython-input-61-3473081656>:2: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
tabla['Color_ojos'] = tabla['Color_ojos'].replace({'Cafe':0,'Negro':1,'Azul':2,'Verde':3})
```

<ipython-input-61-3473081656>:3: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
tabla['Sexo'] = tabla['Sexo'].replace({'M':0,'F':1})
```

Out[61]:      **Color\_ojos** **Sexo** **Edad**

| ID |   |   |    |
|----|---|---|----|
| 1  | 0 | 0 | 24 |
| 2  | 1 | 0 | 23 |
| 3  | 2 | 1 | 31 |
| 4  | 1 | 1 | 27 |
| 5  | 1 | 1 | 24 |
| 6  | 3 | 0 | 28 |
| 7  | 0 | 0 | 25 |

Vamos a resolver **los incisos b), c) y d)**. Pero para este ocasion, vamos agilizar los calculos con un paqueteria que tiene ya definido estas distancias y lo estariamos expresando los resultados en una tabla...

```
In [62]: # Traemos la funcion de "distance"
from scipy.spatial import distance

# Establecemos Los registros como puntos
p1 = tabla.loc[1]
p2 = tabla.loc[5]
p3 = tabla.loc[4]
p4 = tabla.loc[7]
```

- **Distancia Euclidiana (funcion euclidiana):**

```
In [63]: # Calculamos Las distancias
d_euclidiana_1 = [distance.euclidean(p1, p2)]
d_euclidiana_2 = [distance.euclidean(p3, p4)]
```

- **Distancia Manhattan(funcion cityblock):**

```
In [64]: # Calculamos Las distancias
d_manhattan_1 = [distance.cityblock(p1, p2)]
d_manhattan_2 = [distance.cityblock(p3, p4)]
```

- **Distancia Maxima(funcion Chebyshev):**

```
In [65]: # Calculamos Las distancias
d_Maxima_1 = [distance.chebyshev(p1, p2)]
d_Maxima_2 = [distance.chebyshev(p3, p4)]
```

Comparamos las distancias entre si:

```
In [66]: Compa = pd.DataFrame({'Distancia_Euclidiana':[d_euclidiana_1,d_euclidiana_2], 'Distancia_Manhattan':d_euclidiana_1, 'Distancia_Maxima':d_euclidiana_2})
```

```
Out[66]:
```

|   | Distancia_Euclidiana | Distancia_Manhattan | Distancia_Maxima |
|---|----------------------|---------------------|------------------|
| 0 | [1.4142135623730951] | [2]                 | [1]              |
| 1 | [2.449489742783178]  | [4]                 | [2]              |

El **inciso e)** nos indica el normalizar los datos de la edad

```
In [67]: tabla['Edad_normalizada'] = (tabla['Edad']-tabla['Edad'].min())/(tabla['Edad'].max()-tabla['Edad'].min())
```

```
Out[67]:
```

| ID | Color_ojos | Sexo | Edad | Edad_normalizada |
|----|------------|------|------|------------------|
| 1  | 0          | 0    | 24   | 0.125            |
| 2  | 1          | 0    | 23   | 0.000            |
| 3  | 2          | 1    | 31   | 1.000            |
| 4  | 1          | 1    | 27   | 0.500            |
| 5  | 1          | 1    | 24   | 0.125            |
| 6  | 3          | 0    | 28   | 0.625            |
| 7  | 0          | 0    | 25   | 0.250            |

## Sexto Problema

Con los datos anteriores:

- a) Calcula/genera la matriz de correlación de datos. Concluye tus resultados.

### Solucion:

Para este punto sabemos lo que es **Correlacion** y dado a que nos pide la matriz vamos a codificarla *(junto con si Grafica de calor para observar al detalle)..*

```
In [68]: # Con La Correlacion de Pearson y dejando a un lado "La edad"
matriz = tabla.drop('Edad', axis = 1).corr(method = 'pearson')
matriz.round(3)
```

Out[68]:

|                  | Color_ojos | Sexo  | Edad_normalizada |
|------------------|------------|-------|------------------|
| Color_ojos       | 1.000      | 0.167 | 0.661            |
| Sexo             | 0.167      | 1.000 | 0.441            |
| Edad_normalizada | 0.661      | 0.441 | 1.000            |

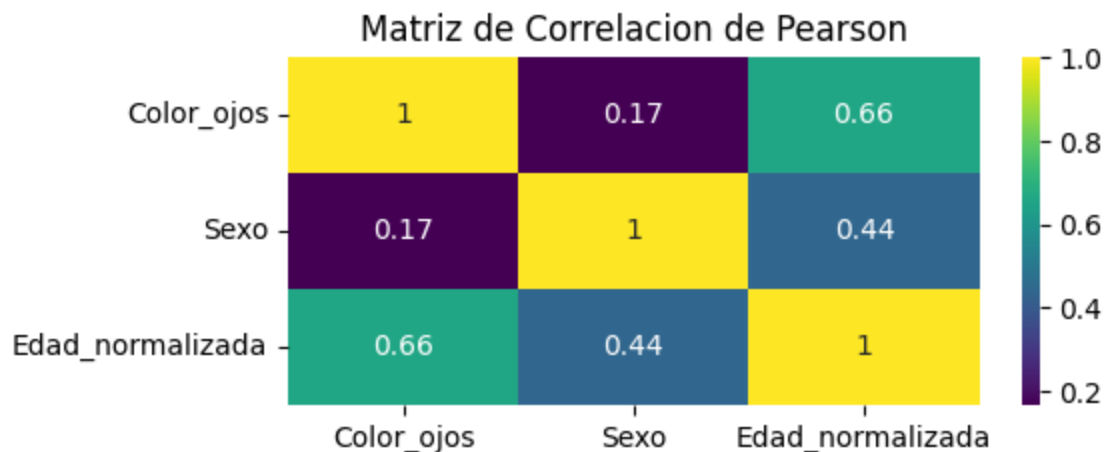
Pasando directamente con la grafica de calor..

```
In [69]: # Creamos el lienzo para la grafica
plt.figure(figsize = (6, 2.5))

# Sobre la gráfica de calor
sns.heatmap(matriz, annot = True, cmap = 'viridis')

# Titulo de la grafica
plt.title('Matriz de Correlacion de Pearson')

plt.tight_layout()
plt.show()
```



Algunas conclusiones que podemos ver son:

- La mejor relacion entre V.A. son entre **la edad** y **el color de ojos** ya que ambas tiene un crecimiento positivo entre ambas del 0.66.
- La peor relacion entre V.A. son entre **Sexo** y **el color de ojos** ya que ambas tiene un crecimiento positivo muy tenue entre ambas del 0.17 (*cercano a 0, casi no tiene alguna relacion*).

Pareciera que ambas correlaciones, no nos proporciona mucha informacion relecante; mas el hecho de la poca intuicion que tenemos sobre que tiene que ver el color de los ojos con la edad e sexo de los registro...

Eso ya recai mas en la coherencia de los datos. Por otra parte podemos crea un modelo predictivo para determinar una valor aproximado en base a los demas V.A. y con ello

podemos ver la relacion intrinseca....

- b) Elaborar un modelo de regresión lineal, expresión lineal. Concluye tus resultados.

### Solucion:

Un modelo de **Regresión lineal** sera una expresion matematica que estara determinado una pendiente central (*Esperanza condicional matematica de  $Y|x$* ) en los datos y con ella podemos dar predecir el valor especifico (V.A. *Dependiente*) en base a otros valores (V.A. *Explicativa*).

Para este caso; matematicamente la recta de regresion sera de la forma:

$$Y = \beta_0 + x_1\beta_1 + x_2\beta_2 + e$$

Donde las betas sera conocida como: **Coeficiente de x** y hay veces en la que analizar esto terminos, se puede determinar si hay (o no) del descarta una V.A.

```
In [70]: # Traemos la paqueteria diseñada para este tipo de modelos
import statsmodels.api as sm

# Preparamos el modelo de regresion lineal por el metodo de Los "Minimos Cuadrado O
modelo = sm.OLS(tabla['Color_ojos'], sm.add_constant(tabla[['Edad_normalizada', 'Se

# Ajustamos el modelo con los datos
resultados = modelo.fit()
```

Para este caso vamos proponer como:

- V.A. DEPENDIENTE Y = "Color\_ojos"
- V.A. EXPLICATIVA X = "Edad\_normalizada", "Sexo"

Por el simple hecho de que la matriz de correlacion anterior arrojaba buenos resultados con esta V.A.

y para averiguar si nuestro modelo es eficiente o no, por medio de un analisis rapido del **Summary...**

```
In [71]: print(resultados.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:                Color_ojos    R-squared:                0.457
Model:                        OLS           Adj. R-squared:          0.185
Method:                       Least Squares  F-statistic:             1.683
Date:                         Wed, 18 Jun 2025  Prob (F-statistic):      0.295
Time:                         05:10:43      Log-Likelihood:          -7.7238
No. Observations:              7            AIC:                    21.45
Df Residuals:                  4            BIC:                    21.29
Df Model:                      2
Covariance Type:               nonrobust
=====
                                coef    std err          t      P>|t|      [0.025    0.975]
-----
const                0.4483    0.574        0.781    0.478    -1.144    2.041
Edad_normalizada     2.2069    1.241        1.778    0.150    -1.240    5.654
Sexo                 -0.3103    0.821       -0.378    0.725    -2.590    1.969
=====
Omnibus:                nan    Durbin-Watson:           2.199
Prob(Omnibus):          nan    Jarque-Bera (JB):        0.531
Skew:                   0.201    Prob(JB):                0.767
Kurtosis:               1.712    Cond. No.                 4.35
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

/usr/local/lib/python3.11/dist-packages/statsmodels/stats/stattools.py:74: ValueWarning: omni_normtest is not valid with less than 8 observations; 7 samples were given.
  warn("omni_normtest is not valid with less than 8 observations; %i "

```

Hay muchas conclusiones que podemos sacar de este resumen del modelo, pero lo haremos parte por parte...

## Sobre los coeficientes

Matemáticamente tenemos que la pendiente de regresión se ve de la forma:

$$Y = 0.4483 + 2.2069 \cdot x_1 + -0.3103 \cdot x_2 + e$$

El análisis sobre el estado de los coeficientes se parte en 2 pruebas:

- **Una prueba de hipótesis:** Indica si es significativo el coeficiente o no en el modelo de regresión; y eso se realiza por medio de una *P-Value* ( $P > |t|$ ). Diremos que si vale la pena utilizar si es menor al 0.05. Pero como no es el caso. Se recomienda ver otros datos.
- **Intervalo de confianza:** Será el conjunto de valores que puede tomar el coeficiente y podemos estar seguros que el coeficiente es significativo si no contiene al cero (*pero también falla en este aspecto*).

## Sobre el error

En la ecuación se vio una V.A.  $e$  que no es más que el error que sería la diferencia entre el valor real y el valor estimado; es decir,

$$e = \hat{Y} - Y$$

Podemos decir que el error no interviene o afecta a los resultados será si este **ruido blanco** que esta V.A. se comporta como la distribución normal de la forma:  $N(0, \sigma)$ .

Podemos ver que tiene este comportamiento por partes:

- **su gráfica de la distribución:** Con los coeficientes (*sesgo debe de estar 0 o cerca*) **Skew: 0.201** y (*Curtosis que de estar cerca del 3*) **Kurtosis: 1.712**. Por ende, no tiene comportamiento de una normal.
- **Prueba de hipótesis de Normalidad:** Para esta prueba jarque-bera y es muy parecida a la prueba anterior y por ende dado a que no es menor al 0.05 podemos concluir que el error no se aproxima a una distribución normal.

## Conclusion en General

Podemos decir que el modelo no es eficiente como sistema predictivo tanto en sus aspectos más fundamentales como en su  $R^2$  (*Coefficiente de determinación*), que no es más el parámetro definitivo de cual nos indica el en las el modelo se ajuste a los datos. Si el nuestro es del 45.6 y por ello debemos de buscar más datos que nos proporcionen más información, al igual que dar otros parámetros de eficiencia (*partiendo los datos en entrenamiento y test*)